

RNA_parte_1

YofreH

2025-09-19

Descenso Gradiente estocástico

El entrenamiento de una red neuronal puede formularse como un **problema de optimización sin restricciones**, pero con una estructura muy específica. Vamos a **reformular el planteamiento general** con todas las variables definidas y ejemplos concretos de la función de pérdida.

Planteamiento matemático general

El entrenamiento de una red neuronal (RNA) consiste en **encontrar los parámetros** que minimicen la pérdida promedio sobre un conjunto de datos de entrenamiento. Formalmente:

\$ \$

* Definición de cada símbolo

Símbolo	Significado matemático	Interpretación en una RNA
$\mathbf{W} \in \mathbb{R}^{d \times p}$	Vector de parámetros (dimensión (d))	Todos los pesos y sesgos de las capas de la red
$\mathbf{x}_i \in \mathbb{R}^p$	Vector de características de entrada	Ejemplo de entrada (imagen, vector numérico, texto codificado)
$y_i \in \mathcal{Y}$	Etiqueta o salida deseada	Clase, valor o distribución objetivo
$f_{\mathbf{W}}: \mathbb{R}^p \rightarrow \mathcal{Y}$	Función implementada por la red con parámetros (W)	El modelo ; composición de capas no lineales
$\ell(\mathbf{y}_i, f_{\mathbf{W}}(\mathbf{x}_i))$	Función de pérdida individual	Mide el error entre predicción y valor real
n	Tamaño del conjunto de entrenamiento	Número total de ejemplos usados

¿Por qué es un problema sin restricciones?

No se imponen condiciones explícitas como $g(\mathbf{W})=0$ o $h(\mathbf{W}) \leq 0$; el objetivo es simplemente minimizar $\mathcal{L}(\mathbf{W})$ sobre todo el espacio de parámetros.

Sin embargo, sí existen **restricciones implícitas**:

- Los pesos (W) están determinados por las **dimensiones de las capas**.
- Las **funciones de activación** (ReLU, Sigmoid, etc.) introducen **no linealidad**.
- En la práctica, se usa **regularización** para evitar soluciones no deseadas (esto se incorpora al funcional, no como restricción).

Por tanto, el problema es **sin restricciones explícitas**, pero **estructuralmente no convexo** y de **alta dimensión**.

Ejemplos comunes de funciones de pérdida ℓ

1. Regresión (salida real)

$$\ell(f_{\mathbf{W}}(\mathbf{x}_i), y_i) = \frac{1}{2} |f_{\mathbf{W}}(\mathbf{x}_i) - y_i|^2$$

Mide el error cuadrático medio (MSE). Ejemplo: predecir temperatura, precio o valor continuo.

2. Clasificación binaria (salida en $[0, 1]$)

$$\ell(f_{\mathbf{W}}(\mathbf{x}_i), y_i) = -y_i \log f_{\mathbf{W}}(\mathbf{x}_i) - (1 - y_i) \log(1 - f_{\mathbf{W}}(\mathbf{x}_i))$$

Es la **entropía cruzada binaria**, usada en redes con activación **sigmoide** en la salida.

3. Clasificación multiclase (salida softmax)

$$\ell(f_W(x_i), y_i) = - \sum_{k=1}^K y_{ik} \log \big(f_W(x_i)_k \big)$$

Donde (K) es el número de clases y y_{ik} es 1 si el ejemplo pertenece a la clase (k). Usada en redes con **softmax** en la salida (p. ej. MNIST).

4. Con regularización L2 incorporada

$$\mathcal{L}_{\text{reg}}(W) = \frac{1}{n} \sum_{i=1}^n \ell(f_W(x_i), y_i) + \lambda \sum_{\ell} \|W^{(\ell)}\|_2^2$$

Aquí ($\lambda > 0$) controla la penalización de pesos grandes. En Keras:
`kernel_regularizer=regularizers.l2( $\lambda$ )`.

Naturaleza del problema

- **No convexo:** la composición de activaciones no lineales crea múltiples mínimos locales.
- **Diferenciable casi en todo punto:** permite el uso de métodos de gradiente (SGD, Adam, RMSprop).
- **De gran dimensión:** (d) puede ser (10^5) a (10^8).
- **Estocástico:** el gradiente se aproxima con minibatches.

Analogía geométrica

En el espacio de parámetros (W), la función $\ell(W)$ define una “superficie de error”. El método de descenso de gradiente estocástico (SGD) sigue una **trayectoria ruidosa descendente** sobre esa superficie hasta llegar a una **región plana de mínima pérdida promedio**.

Gradiente estimado (estocástico)

Para reducir el costo, se toma un **minibatch aleatorio** $B_t \subset \{1, \dots, n\}$ de tamaño $|B_t| = m \ll n$. El gradiente se estima como:

$$\boxed{g_t = \frac{1}{m} \sum_{i \in B_t} \nabla_W \ell(f_W(x_i), y_i)}$$

Este vector g_t es una **aproximación aleatoria** de $\nabla \mathcal{L}(W_t)$.

¿Qué es exactamente un *minibatch*?

- Sea el dataset fijo $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^n$
- Definimos los **índices** $\{1, \dots, n\}$ y, en la iteración t , elegimos un **subconjunto aleatorio** $B_t \subset \{1, \dots, n\}$ de tamaño $|B_t| = m$.

Dos esquemas estándar de muestreo (ambos discretos y uniformes):

1. **Con reemplazo (i.i.d.):** elegimos $(I_1, \dots, I_m)$ i.i.d. con $\mathbb{P}(I_j = i) = 1/n$. Aquí B_t es un **multiconjunto** (puede haber repetidos).
2. **Sin reemplazo (simple random sample):** elegimos B_t **uniforme** entre todas las $\binom{n}{m}$ combinaciones; no hay repetidos dentro del minibatch.

En práctica, es muy común “**random reshuffling**”: en cada época se toma una permutación aleatoria de $\{1, \dots, n\}$ y se parte en bloques de tamaño m . A efectos de **esperanza condicional**, las conclusiones de abajo siguen valiendo.

3. Valor esperado y ruido estocástico

(a) Valor esperado

Si los minibatches B_t se eligen **de forma aleatoria e independiente**, cada muestra x_i tiene la misma probabilidad de ser incluida.

Entonces el **valor esperado** (esperanza matemática) del gradiente estocástico es igual al gradiente real:

$$\mathbb{E}_{B_t}[g_t] = \nabla \mathcal{L}(W_t)$$

es decir, **en promedio**, el gradiente estocástico apunta en la **misma dirección** que el gradiente verdadero.

(b) Ruido estocástico

Definimos el **ruido de gradiente** como la diferencia entre el gradiente estocástico y el gradiente exacto:

$$\xi_t = g_t - \nabla \mathcal{L}(W_t)$$

Por construcción:

$$\mathbb{E}[\xi_t] = 0, \quad \text{Cov}[\xi_t] = \Sigma_t \neq 0.$$

- ξ_t es un **término de ruido aleatorio** que refleja la variabilidad por muestreo.
- Su varianza Σ_t **disminuye** cuando aumenta el tamaño del minibatch m .
 - Si $m=n$ (todo el dataset), entonces $\xi_t=0 \Rightarrow$ método determinista (GD).
 - Si $m=1$, el ruido es máximo $\Rightarrow$ método “puro” SGD.

¿Por qué g_t es aleatorio? ¿Cuál es su esperanza?

Definamos el **gradiente por dato** en (W_t) $g_i(W_t) = \nabla \mathcal{L}(W_t; x_i, y_i)$, $\mathcal{L}(W_t) = \frac{1}{n} \sum_{i=1}^n \mathcal{L}(W_t; x_i, y_i)$

El **gradiente estocástico** (minibatch) es $g_t = \frac{1}{m} \sum_{i \in B_t} g_i(W_t)$ (y con reemplazo sería $\frac{1}{m} \sum_{j=1}^m g_{I_j}(W_t)$)

- Es aleatorio** porque B_t (o los I_j) se eligen **al azar**.
- Imparcialidad (unbiasedness)**, condicional en (W_t) : $\mathbb{E}[g_t | W_t] = \nabla \mathcal{L}(W_t)$. *Prueba (sin reemplazo)*: por simetría, $\mathbb{P}(i \in B_t) = m/n$. $\mathbb{E}[\frac{1}{m} \sum_{i \in B_t} g_i(W_t) | W_t] = \frac{1}{m} \sum_{i=1}^n \mathbb{P}(i \in B_t) g_i(W_t) = \frac{m}{n} \cdot \frac{1}{m} \sum_{i=1}^n g_i(W_t) = \nabla \mathcal{L}(W_t)$. Con reemplazo, el cálculo es idéntico usando $\mathbb{E}[g_{I_j}] = \frac{1}{n} \sum_i g_i$.

Ruido estocástico. Definimos $\xi_t = g_t - \nabla \mathcal{L}(W_t)$, entonces $\mathbb{E}[\xi_t | W_t] = 0$. Ese es el **ruido**: la variación extra debida al muestreo.

Covarianza condicional del ruido. Es útil cuantificar la dispersión:

- Con **reemplazo (i.i.d.)**: $\text{Cov}(g_t, | W_t) = \frac{1}{m} \Sigma(W_t)$, $\Sigma(W_t) = \text{Cov}(\xi_t) \sim \text{Unif}[n] \cdot \text{Cov}(g_i(W_t))$.
- Sin reemplazo** (muestra simple): $\text{Cov}(g_t, | W_t) = \frac{1}{m} \text{Cov}(1 - \frac{m}{n} \text{Cov}(W_t))$, donde $\text{Cov}(W_t) = \frac{1}{n(n-1)} \sum_{i=1}^n \text{Cov}(g_i(W_t) - \nabla \mathcal{L}(W_t))$ (aparece el **factor de corrección por población finita** $(1 - m/n)$).

Consecuencia: aumentar m **reduce la varianza** del gradiente (más estabilidad).

¿El ruido ξ_t sigue una distribución normal? ¿Cuál es la justificación?

- En general, no.** Para minibatches pequeños, la distribución de g_t es **discreta** (mezcla finita de valores) determinada por las combinaciones posibles; no tiene por qué ser gaussiana.
- Aproximación normal (CLT)**: si muestreamos **con reemplazo** y m es **moderado/grande**, g_t es un **promedio** de m vectores i.i.d.; bajo condiciones estándar (momentos finitos, Lindeberg), por el **Teorema Central del Límite**:

$$\sqrt{m} g_t \xrightarrow{d} \mathcal{N}(0, \Sigma(W_t))$$

Sin reemplazo, existe un CLT análogo con el **factor de población finita**. En práctica, esta normalidad es **aproximada** y mejora al crecer m .

(c) Interpretación probabilística

El SGD puede verse como un proceso de **Monte Carlo** que estima la esperanza del gradiente:

$$g_t \approx \mathbb{E}_X[\nabla \mathcal{L}(W_t; X, Y)]$$

Cada iteración usa una **muestra aleatoria** para estimar la dirección promedio de descenso.

Ejemplo numérico paso a paso (unidimensional)

Modelo y pérdida por dato (elegimos $\ell_i = \frac{1}{2}(w x_i - y_i)^2 \Rightarrow g_i(w) = \partial \ell_i / \partial w = x_i(w x_i - y_i)$)

Dataset $(n=3)$ $(x_1, y_1)=(1,2)$; $(x_2, y_2)=(2,4)$; $(x_3, y_3)=(3,6)$. En $w=1.1$: $g_1=-0.9$, $g_2=-3.6$, $g_3=-8.1$. Gradiente **total**: $\nabla \mathcal{L}(w) = \frac{1}{3}(-0.9 - 3.6 - 8.1) = -4.2$.

Mini-batch de tamaño $(m=1)$ (con reemplazo)

- g_t toma valores $(-0.9, -3.6, -8.1)$ cada uno con prob. $(1/3)$.
- $\mathbb{E}[g_t] = (-0.9 - 3.6 - 8.1)/3 = -4.2$ (imparcial).

- $\text{Var}(g_t) = 8.82$ (grande).

Mini-batch de tamaño (m=2) (sin reemplazo)

Los posibles promedios: $\frac{g_1+g_2}{2} = -2.25, \frac{g_1+g_3}{2} = -4.50, \frac{g_2+g_3}{2} = -5.85$, cada uno con probabilidad $1/3$.

- $E[g_t] = (-2.25 - 4.50 - 5.85)/3 = -4.2$ (imparcial).
- $\text{Var}(g_t) = 2.205$

Comprueba la teoría de varianzas Aquí Sea la “covarianza poblacional” escalar $\Sigma = \frac{1}{n-1} \sum_{i=1}^n (g_i - \bar{g})^2 = 26.46 = 13.23$, con $\bar{g} = -4.2$. Entonces $\text{Var}(g_t) = \frac{1}{m} \text{Big}(1 - \frac{1}{m}) \Sigma = \frac{1}{2} \cdot \frac{1}{3} \cdot 13.23 = 2.205$ (coincide con el cálculo directo). Observa cómo **crecer (m)** reduce la varianza.

(d) Ecuación con ruido explícito

La actualización de los parámetros se puede escribir como:

$$W_{t+1} = W_t - \eta \nabla_{\mathcal{L}(W_t)} \xi_t, \quad E[\xi_t | W_t] = 0, \quad \text{Cov}(\xi_t | W_t) \approx \frac{1}{m} \Sigma(W_t)$$

- $-\eta \nabla_{\mathcal{L}(W_t)}$: movimiento “promedio” de descenso.
- $-\eta \xi_t$: **sacudida aleatoria** por muestreo; decrece al aumentar (m); puede ayudar a escapar de mínimos agudos.
- Si (m) es grande (y condiciones regulares), ξ_t es **aprox. normal** por CLT; si (m) pequeño, su distribución es **discreta/no gaussiana**.

Esta ecuación muestra que **SGD es un proceso aleatorio**, y puede modelarse como una **ecuación diferencial estocástica discreta** (lo veremos en la conexión continua).

Conclusión conceptual

- El caso determinista corresponde a una **ecuación en diferencias**: $W_{t+1} = W_t - \eta \nabla_{\mathcal{L}(W_t)}$, que es la **discretización de una ODE** $\frac{dW}{dt} = -\nabla_{\mathcal{L}(W)}$. Su trayectoria es **suave**.
- El caso estocástico introduce **ruido aditivo**: $W_{t+1} = W_t - \eta \nabla_{\mathcal{L}(W_t)} + \xi_t, \quad E[\xi_t] = 0, \quad \text{Cov}(\xi_t) = \Sigma/m$. Es la **discretización de una EDE (ecuación diferencial estocástica)**: $dW_t = -\nabla_{\mathcal{L}(W_t)} dt + \sqrt{2D} dB_t$. Su trayectoria es **aleatoria**, pero la **esperanza** sigue la misma dirección de descenso.

Determinista vs estocástico

no se trata solo de eficiencia, sino también de **propiedades matemáticas y de generalización** del modelo.

comparando ambos métodos: descenso **determinista (GD)** vs. **estocástico (SGD)**, y definiendo **cuándo conviene usar cada uno**.

1. Recordatorio formal

- **Descenso de gradiente determinista (GD)**: $W_{t+1} = W_t - \eta \nabla_{\mathcal{L}}(W_t), \quad \mathcal{L}(W) = \frac{1}{n} \sum_{i=1}^n \ell(f_W(x_i), y_i)$
- **Descenso de gradiente estocástico (SGD)**: $W_{t+1} = W_t - \eta g_t, \quad g_t = \frac{1}{m} \sum_{i \in B_t} \nabla_{W_t} \ell(f_W(x_i), y_i)$, donde B_t es un **minibatch aleatorio** de tamaño $m \ll n$

2. Diferencias esenciales

Propiedad	Determinista (GD)	Estocástico (SGD)
Tipo de gradiente	Exacto (usa todos los datos)	Aproximado (usa muestra aleatoria)
Costo por iteración	Alto ($O(n)$)	Bajo ($O(m)$)
Dirección de descenso	Precisa	Ruidosa, variable
Trayectoria	Suave	Zigzagueante
Convergencia	Monótona, estable	Oscilatoria, pero rápida en promedio
Ruido	Ninguno	Presente: $\xi_t = g_t - \nabla_{\mathcal{L}}(W_t)$
Análogo	Ecuación diferencial determinista	Ecuación diferencial estocástica (tipo Langevin)

3. Cuándo se recomienda SGD (estocástico)

a) Dataset grande (n grande)

- Si el conjunto de datos es enorme (miles o millones de ejemplos), calcular el gradiente exacto es **computacionalmente prohibitivo**.
- SGD reduce el costo de cada actualización de $O(n)$ a $O(m)$, con $m \ll n$. Permite **entrenar redes profundas** en tiempo razonable.

b) Superficies de pérdida no convexas (redes neuronales profundas)

- En redes profundas, la función de pérdida $\mathcal{L}(W)$ tiene **múltiples mínimos locales y valles planos**.
- El **ruido del gradiente** (ξ_t) ayuda a:
 - escapar de mínimos locales agudos,
 - explorar regiones planas,
 - encontrar soluciones con mejor generalización (mandt et al., ICLR 2017).

En términos de ecuaciones diferenciales estocásticas:

$$dW_t = -\nabla \mathcal{L}(W_t)dt + \sqrt{2D}dB_t,$$

donde D depende de la varianza del minibatch.

c) Modelos con alto costo de gradiente

- Cuando el cálculo del gradiente completo es costoso (por ejemplo, en **redes recurrentes** o **transformers grandes**), se prefiere estimar el gradiente parcial.

d) Datos que llegan en flujo continuo (online learning)

- En contextos donde los datos llegan progresivamente (por ejemplo, sensores, streaming, o trading), el SGD puede **actualizar los parámetros en tiempo real** sin almacenar todo el dataset.

e) Mejora de la generalización

- En problemas de aprendizaje profundo, el **ruido del SGD** actúa como **regularizador implícito**:
 - evita sobreajuste,
 - selecciona soluciones más planas,
 - mejora la robustez frente a perturbaciones en los datos.

Matemáticamente: el ruido de SGD induce una distribución estacionaria alrededor de los mínimos anchos, no agudos, lo que correlaciona con una menor curvatura del Hessiano y, por tanto, mejor generalización.

4. Cuándo se prefiere GD (determinista)

Escenario	Justificación
Dataset pequeño o mediano	No hay ventaja de muestrear; puedes usar todos los datos sin costo excesivo.
Función de pérdida convexa (p. ej. regresión lineal simple)	El GD converge de forma más estable y exacta.
Necesidad de precisión numérica alta	El ruido de SGD puede impedir convergencia exacta.
Análisis teórico o validación formal	GD tiene una dinámica determinista más fácil de analizar (por ejemplo, en convexidad estricta).

5. Caso intermedio: Mini-Batch SGD

En práctica, se usa una combinación: $W_{t+1} = W_t - \eta_t g_t, \quad |B_t| = m \in [16, 1024].$

Esto **balancea**:

- Ruido suficiente para explorar (mejor generalización),

(c) Bias de salida Escalar Desplaza verticalmente la salida Mueve la curva arriba o abajo

Derivada: $u_{\theta}'(x) = v, w, \operatorname{sech}^2(wx+b).$

Residuo de la EDO (lo que debería ser 0): $r_{\theta}(x) := u_{\theta}'(x) + u_{\theta}(x) = v, w, \operatorname{sech}^2(s) + v, \tanh(s) + c, \text{quad } s := wx+b.$

4) Pérdida (PINN “suave” con penalización)

Con N puntos interiores $\{x_k\}$ y una penalización $(\lambda > 0)$ para la condición inicial: $\mathcal{L}(\theta) = \frac{1}{N} \sum_{k=1}^N r_{\theta}(x_k)^2 + \lambda (u_{\theta}(0) - 1)^2, \text{quad } u_{\theta}(0) = v, \tanh(b) + c.$

5) Gradientes analíticos (para ver “cómo aprende”)

Es útil anotar las derivadas parciales del **residuo** $r_{\theta}(x)$ y de la **condición** $u_{\theta}(0)$.

Sea $t := \tanh(s)$ y $q := \operatorname{sech}^2(s) = 1 - t^2$. Entonces $\begin{aligned} \frac{\partial r}{\partial v} &= w, q + t, \frac{\partial r}{\partial w} = v, q - 2vwx, q, t + v, x, q = v, q(1+x) - 2vwx, q, t, \frac{\partial r}{\partial b} = -2vwx, q, t + v, q, \frac{\partial r}{\partial c} = 1. \end{aligned}$

Para la condición inicial $u_{\theta}(0) = v, \tanh(b) + c$: $\frac{\partial u}{\partial v} = \tanh(b), \text{quad } \frac{\partial u}{\partial w} = 0, \text{quad } \frac{\partial u}{\partial b} = v, \operatorname{sech}^2(b), \text{quad } \frac{\partial u}{\partial c} = 1.$

Regla de la cadena para $\mathcal{L}$: $\frac{\partial \mathcal{L}}{\partial v} = \frac{2}{N} \sum_{k=1}^N r_{\theta}(x_k), \frac{\partial \mathcal{L}}{\partial w} = \frac{2}{N} \sum_{k=1}^N r_{\theta}(x_k) \frac{\partial r}{\partial w} + 2\lambda (u_{\theta}(0) - 1) \frac{\partial u}{\partial w}, \frac{\partial \mathcal{L}}{\partial b} = \frac{2}{N} \sum_{k=1}^N r_{\theta}(x_k) \frac{\partial r}{\partial b} + 2\lambda (u_{\theta}(0) - 1) \frac{\partial u}{\partial b}, \frac{\partial \mathcal{L}}{\partial c} = \frac{2}{N} \sum_{k=1}^N r_{\theta}(x_k) \frac{\partial r}{\partial c} + 2\lambda (u_{\theta}(0) - 1) \frac{\partial u}{\partial c}.$

En práctica, esto lo hace *autodiff*; aquí se ve que la PINN **minimiza** el residuo interior y la violación de la condición inicial.

6) Mini-cálculo numérico en un punto (todo a mano)

Toma parámetros “plausibles” que ya muestran decaimiento: $v = -1, \text{quad } w = 1, \text{quad } b = 0, \text{quad } c = 1.$ En $(x = 0.5): (s = 0.5), (t = \tanh(0.5) \approx 0.462117), (q = \operatorname{sech}^2(0.5) \approx 0.786447).$

- **Aproximación y derivada:** $u_{\theta}(0.5) = v, t + c = -0.462117 + 1 = 0.537883, \text{quad } u_{\theta}'(0.5) = v, w, q = -0.786447.$
- **Residuo:** $r_{\theta}(0.5) = u_{\theta}'(0.5) + u_{\theta}(0.5) = -0.786447 + 0.537883 = -0.248564.$
- **Gradientes del residuo** en $(x = 0.5)$: $\begin{aligned} \frac{\partial r}{\partial v} &= w, q + t = 0.786447 + 0.462117 = 1.248564, \frac{\partial r}{\partial w} = v, q(1+x) - 2vwx, q, t = -0.786447(1.5) + (-1) \cdot 0.786447 \cdot 0.5 \cdot 0.462117 \\ &= -1.179671 + 0.363000 \approx -0.816671, \frac{\partial r}{\partial b} = v, q - 2vwx, q, t = -0.786447 + 0.726000 = -0.060447, \frac{\partial r}{\partial c} = 1. \end{aligned}$
- **Condición inicial:** $u_{\theta}(0) = v, \tanh(b) + c = 1 \Rightarrow$ el término de contorno **no aporta** a $\nabla \mathcal{L}$ (su residuo es 0).

Si (sólo para ilustrar) usas este **único** punto interior ($N = 1$), (λ) arbitraria), los gradientes de $\mathcal{L}$ quedan: $\frac{\partial \mathcal{L}}{\partial v} = 2, r, \frac{\partial \mathcal{L}}{\partial w} = 2(-0.248564)(1.248564) = -0.620, \text{quad } \frac{\partial \mathcal{L}}{\partial b} = 2(-0.248564)(-0.816671) = +0.406, \text{quad } \frac{\partial \mathcal{L}}{\partial c} = 2(-0.248564)(-0.060447) = +0.030, \text{quad } \frac{\partial \mathcal{L}}{\partial \lambda} = 2(-0.248564) \cdot 1 = -0.497.$

Una **pasada de descenso** $(\theta \leftarrow \theta - \eta \nabla \mathcal{L})$ movería (w) y (b) **hacia arriba** (signos $(+)$), y (v, c) **hacia arriba** también (porque restas un gradiente negativo), todo con el fin de **reducir el residuo** en el punto.

7) Variante con restricción dura (condición inicial exacta)

Otra táctica frecuente en PINNs: construir un ansatz que **satisfaga la condición** por construcción. Aquí, impón $u(0) = 1$ mediante $\widehat{u}_{\theta}(x) = 1 + x, N_{\theta}(x),$ con (N_{θ}) una red cualquiera. Entonces $\widehat{u}_{\theta}(0) = 1$ **siempre**. El residuo es $\widehat{r}_{\theta}(x) = \widehat{u}_{\theta}'(x) + \widehat{u}_{\theta}(x) = \text{big}(N_{\theta}(x) + x, N_{\theta}'(x)) + \text{big}(1 + x, N_{\theta}(x)),$ y la pérdida **ya no necesita** término de contorno: $\mathcal{L}(\theta) = \frac{1}{N} \sum_{k=1}^N \widehat{r}_{\theta}(x_k)^2.$ (De nuevo, (N_{θ}) se obtiene por regla de la cadena —*autodiff* en práctica—, pero aquí ves la **forma exacta** de las derivadas que se minimizan.)

Conclusión

- Una **PINN** minimiza el **residuo** de la EDO en puntos interiores y, además, **impone** las **condiciones** (suaves o duras).
- El ejemplo muestra, con **fórmulas explícitas**, cómo se computan $\mathcal{L}(\theta)$ y sus **gradientes**.
- Con suficientes puntos y una red un poco más rica, el mínimo de $\mathcal{L}$ aproxima la solución $u(x)=e^{-x}$ en $[0,1]$.

¿Quieres que haga lo mismo para una **EDO de 2º orden** (por ejemplo, $u''+u=0$ con $u(0)=0, u'(0)=1$) para que veas cómo entra $u''_{\theta}(x)$ en el residuo?